rsedit — Rapporto sui problemi del codice

Le schede dell'analisi della revisione `08a853d`, verificate sulla revisione `3d846f1`

Indice

1 Che cosa contiene questo rapporto 2
1.1 Le schede dell'interprete 2
1.2 Come leggere una scheda 2
1.3 Come è stato condotto l'esame 2
1.4 Riepilogo 3
2 Buffer, modifiche e annullamento 6
3 Tastiera, comandi e primitive dell'editor 8
4 Minibuffer 11
5 File e salvataggio 13
6 Avvio, configurazione e registro 15
7 Lavoro in background 17
8 Modi, sintassi e colorazione 18
9 Ricerca, sostituzione e liste di risultati 21
10 Comandi di shell 24
11 Disegno e terminale 25
12 La finestra grafica 28
13 Documentazione, commenti, codice morto, build e test 29
14 Appendici 31
14.1 Le schede risolte 31
14.2 Differenze da Emacs che non sono difetti 31
14.3 Parti esaminate e trovate solide 31

1 Che cosa contiene questo rapporto

Questo documento elenca i problemi trovati analizzando il codice di rsedit. È separato di proposito dal manuale di architettura, `architettura-editor.typ`. Il manuale descrive come il codice funziona e resta valido anche dopo una correzione; questo rapporto descrive dove il codice non fa ciò che dovrebbe. Quando un problema viene risolto, la sua scheda passa nell'appendice delle schede risolte, con il commit che lo ha risolto.

Ogni problema ha un identificativo stabile, formato dalla sigla dell'area e da un numero (`BUF-2`, `CMD-1`…), che si può citare in un commit o in una discussione; nel codice lo si cita solo in un `TODO(ID)`, e solo finché la scheda è aperta. Le schede sono generate da un'unica tabella di dati all'inizio del sorgente: per aggiungerne una, o per segnarne una come risolta, basta modificare quella tabella, e il riepilogo si aggiorna da solo.

1.1 Le schede dell'interprete

Le schede delle aree LET (il lettore) e INT (il valutatore e la libreria di base) riguardano l'interprete, che da `6d30567` è il crate esterno `risp`. Sono passate nel rapporto del suo repository, `doc/rapporto-problemi.typ`, con gli stessi identificativi. Le schede di qui che vi rimandano lo dicono nel testo, per esempio «INT-2, nel rapporto di risp». rsedit usa la revisione di risp fissata nel suo `Cargo.lock`, oggi `b2d000b`: una scheda che risp risolve dopo quella revisione resta un difetto di rsedit finché la dipendenza non si aggiorna.

1.2 Come leggere una scheda

Ogni scheda riporta:

- **Gravità**, secondo la scala qui sotto.
- **Dove**: i file e le funzioni, o i tipi, in cui sta il difetto. I percorsi senza prefisso sono relativi a `core/src/`; quelli che cominciano con `src/` riguardano la TUI e la GUI, quelli con `core/lisp/` i moduli Lisp.
- **Verifica**: «verificato» se il comportamento è stato riprodotto eseguendo il codice (il testo dice come, e su quale revisione), «dal codice» se la conclusione viene dalla sola lettura.
- **Cosa succede**, il meccanismo del difetto; **Come riprodurlo**, quando serve; **Effetto**, ciò che l'utente vede; **Correzione proposta**.

Tutte le schede dell'elenco sono aperte; le risolte sono nell'appendice.

Gravità	Significato
ALTA	L'editor termina, si blocca senza via d'uscita o perde dati; oppure un contenuto esterno può far eseguire azioni al terminale; oppure l'editor finisce in uno stato da cui si esce solo riavviando.
MEDIA	Un comportamento sbagliato o un rallentamento di secondi in un uso realistico, senza perdita di dati.
BASSA	Casi limite, incoerenze, codice morto, commenti e documentazione superati.

1.3 Come è stato condotto l'esame

Il codice della revisione `08a853d` è stato letto per intero: l'interprete (allora `core/src/lisp`, ora il crate `risp`), la libreria dell'editor (`core/src`), la TUI (`src/`) e i moduli Lisp (`core/lisp`). Ogni comportamento sospetto è stato riprodotto, dove possibile, con piccoli programmi esterni che usano `rsedit_core` come una libreria:

- un **esecutore di scenari** che valuta espressioni Lisp, preme tasti attraverso il percorso reale dei tasti, attende i lavori di background e stampa buffer, area messaggi ed evidenziazioni della fotografia dello schermo;

- un **fuzzer delle primitive**, che chiama ciascuna delle 628 funzioni registrate (tranne quelle che toccano file o processi esterni) con 807 combinazioni di argomenti anomali (zero, negativi, 10^{23}, infinito, NaN, stringhe vuote e non ASCII, liste puntate, vettori, mappe, lambda), una volta su un buffer vuoto e una su un buffer con testo multiriga, caratteri non ASCII, tab, una regione attiva e finestre divise.

I fuzzer del lettore e delle forme speciali, che hanno trovato le schede dell'interprete, sono descritti nel rapporto di risp. Le misure di tempo sono state prese con build di release su una macchina condivisa, quindi vanno lette come ordini di grandezza.

1.3.1 La verifica sulla revisione `3d846f1`

Ogni scheda è stata poi verificata sulla revisione `3d846f1`. Il codice che descrive è stato riletto, e la storia di git ha detto quale commit lo ha cambiato. Dove la scheda dice «verificato», la riproduzione è stata eseguita di nuovo, con lo stesso esecutore di scenari o con due editor veri in un terminale virtuale. Le misure di tempo e di memoria non sono state ripetute dove il codice misurato non è cambiato, e la scheda lo dice. Il campo **Dove** nomina ora file e funzioni, non righe.

Delle 51 schede di rsedit, 11 sono risolte. Sette lo sono in parte e restano aperte con il testo aggiornato a ciò che manca: BUF-1, CMD-1, CMD-3, CMD-6, MIN-3, FIL-5 e DOC-1. La verifica ne ha trovate quattro nuove: AVV-6, BKG-2, RIC-7 e DOC-5. Le schede trovate dopo, nei passi della documentazione che riscrivono il codice, dicono nella verifica dove sono state riprodotte, o che vengono dalla lettura del codice: RIC-8, BUF-4, CMD-7, CMD-8, BKG-3, SHL-5, MIN-4, RIC-9, MIN-5, FIL-6, SIN-7, SIN-8, SIN-9, AVV-7, AVV-8, GUI-1 e CMD-9.

1.4 Riepilogo

> Sono aperti **61 problemi**: **4 di gravità alta**, 20 di gravità media e 37 di gravità bassa; 40 sono stati riprodotti eseguendo il codice. Altri 11 sono stati risolti dopo l'analisi, e sono nell'appendice.

Dei quattro problemi di gravità alta, due riguardano il terminale: un file può mandargli sequenze di controllo (TUI-1), e un panic o un errore lo lasciano in raw mode (TUI-2). Gli altri due fanno perdere lavoro senza dirlo: un salvataggio fallito non viene segnalato (FIL-1), e due prompt aperti insieme perdono il testo del primo e lasciano una finestra che non si chiude (MIN-1). Delle tre correzioni trasversali che l'analisi proponeva, una c'è: nel WorkManager un panic ritira solo il lavoro che l'ha causato (BKG-1, risolta). Restano da fare la guardia che ripristini il terminale (TUI-2) e un limite alle quantità guidate dall'utente, nelle primitive dell'editor (CMD-3) come nell'interprete (INT-2 e INT-13, nel rapporto di risp).

La tabella seguente elenca i problemi aperti, dal più grave; l'identificativo rimanda alla scheda.

ID	Problema	Gravità	Verifica
MIN-1	Prompt annidati: testo perso e finestra flottante fantasma	ALTA	verificato
FIL-1	`save-buffer` fallito: solo una riga nel registro	ALTA	verificato
TUI-1	Caratteri di controllo stampati così come sono: iniezione di sequenze nel terminale	ALTA	verificato
TUI-2	Nessun ripristino del terminale su panic o errore	ALTA	dal codice
BUF-1	`undo` / `redo` scavalcano le porte: sola lettura, overlay, testo virtuale	MEDIA	verificato
BUF-2	`GapBuffer::cursor_pos` lineare nella posizione	MEDIA	verificato
CMD-1	Tasti non ASCII: non digitabili con la configurazione predefinita	MEDIA	verificato
CMD-2	`KeyCode` senza Delete/Home/End/PgUp/PgDn/F1…	MEDIA	dal codice
CMD-3	Larghezze e conteggi senza limite nelle primitive Rust: panic e abort	MEDIA	verificato
MIN-2	`C-g` nel minibuffer non chiude il prompt	MEDIA	verificato
MIN-5	`minibuffer-choose-completion` fuori da un prompt riscrive il buffer corrente	MEDIA	verificato

ID	Problema	Gravità	Verifica
FIL-2	Scrittura non atomica dei file	MEDIA	dal codice
FIL-3	Aprire un file non UTF-8 fallisce in silenzio	MEDIA	verificato
AVV-2	`eval_file`: errori di sintassi silenziosi; un commento finale senza a capo annulla il file	MEDIA	verificato
AVV-7	Un errore in `init.lisp` o in un modulo: l'editor stampa l'errore ed esce con 0	MEDIA	verificato
SIN-1	Regioni che corrispondono a «»: ciclo infinito o memoria esaurita nell'evidenziatore	MEDIA	verificato
SIN-2	Colorazione quadratica sulle righe lunghe	MEDIA	verificato
SIN-6	`C-M-f` a metà di un simbolo: un'espressione di troppo	MEDIA	verificato
SIN-8	Alla fine di una lista `forward-sexp` e `down-list` entrano nelle liste dopo	MEDIA	verificato
RIC-1	`replace-regexp` con match vuoti: tutto nello stesso punto	MEDIA	verificato
RIC-2	`C-M-r`: tempo quadratico per tasto	MEDIA	verificato
TUI-3	Tab non espansi: cursore, colori e riempimento sfasati	MEDIA	verificato
TUI-4	CJK ed emoji contati come una colonna	MEDIA	dal codice
TUI-5	`Color::parse`: panic su colori non ASCII	MEDIA	verificato
BUF-3	`GapBuffer::clone` è `todo!()`; `find_forward` underflow	BASSA	dal codice
BUF-4	Sola lettura: kill, verbi, commenti e rettangoli tacciono; `kill-rectangle` conserva il blocco	BASSA	verificato
CMD-4	`parse_key`: un solo prefisso; `define-key` fallisce in silenzio	BASSA	verificato
CMD-5	`C-u 3 x` inserisce una sola `x`	BASSA	verificato
CMD-6	`kill-word`, `delete-char` e simili: un conteggio negativo vale 0	BASSA	dal codice
CMD-7	Windows: `insert-pasted-text` non converte CR e CRLF	BASSA	dal codice
CMD-8	`close-floating-window` senza flottanti: focus su una finestra che può non esserci	BASSA	verificato
CMD-9	vim: entrare e uscire da Insert costa quanto il file	BASSA	verificato
MIN-3	`set_minibuffer_content` scavalca le porte e lascia il testo dopo il punto	BASSA	verificato
MIN-4	MODE nil a `minibuffer-read`: il prompt senza i suoi tasti	BASSA	verificato
FIL-4	Corsa tra copia del testo e azzeramento di «modificato»	BASSA	dal codice
FIL-5	Autosave: versione segnata anche se la scrittura fallisce	BASSA	dal codice
FIL-6	Copie in `auto-save-directory`: `/a!/b` e `/a/!b` hanno lo stesso nome	BASSA	verificato
AVV-5	`init.lisp` predefinito scritto solo alla prima esecuzione	BASSA	dal codice
AVV-6	Due editor insieme: il secondo tronca il registro del primo, e le righe si sovrappongono	BASSA	verificato
AVV-8	Senza cache, niente `rsedit.log`, in silenzio	BASSA	dal codice
BKG-2	Un panic ritira il lavoro per sempre: colorazione o salvataggio automatico fermi senza avviso	BASSA	dal codice
BKG-3	`setenv`: le letture dell'ambiente fuori da `std` non sono escluse	BASSA	dal codice
SIN-4	`make-mode` ricrea il modo, perdendo legami e hook	BASSA	dal codice
SIN-5	Facce: registro globale senza limite, indice `u16`	BASSA	dal codice
SIN-7	GROUP che non partecipa, o che il pattern non ha: tutta la corrispondenza colorata	BASSA	verificato
SIN-9	Commenti a blocchi contati come espressioni, quelli di riga no	BASSA	verificato
RIC-3	Una sostituzione = un passo di `undo`	BASSA	verificato
RIC-4	Match oltre il limite scartato; colonna ricalcolata per ogni match	BASSA	verificato
RIC-5	Risultati per numero di riga, non per marcatore	BASSA	verificato

ID	Problema	Gravità	Verifica
RIC-7	Liste di occur e di grep in due strisce separate	BASSA	verificato
RIC-8	`query-replace` lasciato: sessione ed evidenziazione restano	BASSA	verificato
RIC-9	BUFFER nil a `results-put` e `results-select`: il buffer `nil`	BASSA	verificato
SHL-4	`manpage`: nome interpolato senza quoting nella shell	BASSA	verificato
SHL-5	All'uscita il `KILL` di un comando fermato non parte	BASSA	verificato
TUI-6	Finestre flottanti senza sintassi e overlay	BASSA	dal codice
TUI-7	`%p` = `All` solo con una riga	BASSA	dal codice
TUI-8	Niente schermo alternativo: all'uscita lo schermo viene cancellato	BASSA	dal codice
GUI-1	GUI: un aggiornamento ogni 250 ms anche da ferma	BASSA	dal codice
DOC-1	Commenti e docstring superati	BASSA	dal codice
DOC-2	`Debug` di `EditorState` è `todo!()`, `PartialEq` è `unreachable!()`	BASSA	dal codice
DOC-5	`free_buffer_name` mai usato; due `if` da unire; 41 blocchi da formattare	BASSA	verificato

Area	Descrizione	Alta	Media	Bassa	Risolte
BUF	Buffer, modifiche e annullamento	0	2	2	0
CMD	Tastiera, comandi e primitive dell'editor	0	3	6	0
MIN	Minibuffer	1	2	2	0
FIL	File e salvataggio	1	2	3	0
AVV	Avvio, configurazione e registro	0	2	3	3
BKG	Lavoro in background	0	0	2	1
SIN	Modi, sintassi e colorazione	0	4	4	1
RIC	Ricerca, sostituzione e liste di risultati	0	2	6	1
SHL	Comandi di shell	0	0	2	3
TUI	Disegno e terminale	2	3	3	0
GUI	La finestra grafica	0	0	1	0
DOC	Documentazione, commenti, codice morto, build e test	0	0	3	2

Tabella 1: I problemi aperti per area e gravità, e quelli risolti

2 Buffer, modifiche e annullamento

BUF-1 — `undo` e `redo` modificano il testo senza passare dalle due porte delle modifiche

Gravità	MEDIA
Dove	`primitives/undo.rs` (le primitive `undo` e `redo`), `buffer/undo.rs` (`UndoHistory::undo`, `UndoHistory::redo`, `apply_inverse`), `editor/replace.rs` (`EditorState::replace_back`)
Verifica	Riprodotto sulla revisione `3d846f1`: dopo `(insert "abc")`, `(undo-boundary)` e `(set-buffer-read-only t)`, `(undo)` svuota il buffer senza messaggi; un overlay su `world` in `XXhello world` resta agli offset 8–13 dopo che `undo` ha tolto `XX`. La colorazione e le cache, che l'analisi aveva trovato vecchie dopo `C-/`, ora si aggiornano: lo provano i test `undo_and_redo_recolour_what_they_change` e `an_undo_does_not_leave_a_stale_answer_behind`.

Cosa succede. Ogni modifica dovrebbe passare da `insert_text` o `delete_range`. Sono loro che controllano la sola lettura, incrementano `version`, invalidano le cache di colorazione e di scansione, disattivano il mark e spostano overlay e testo virtuale. `undo` e `redo` applicano invece le modifiche inverse direttamente sul testo (`apply_inverse`). Da `9956bed` chiamano poi `changed`, che incrementa la versione, invalida le cache dalla prima riga toccata ed emette `buffer-changed`; il resto delle due porte resta saltato.

Come riprodurlo.

```
(insert "abc") (undo-boundary)
(set-buffer-read-only t)
(undo)                          ; il buffer si svuota, senza «Buffer is read-only»
```

Effetto. (1) `undo` modifica i buffer in sola lettura. (2) Overlay e testo virtuale restano agli offset del testo vecchio, quindi le evidenziazioni cadono sulle parole sbagliate.

Correzione proposta. Far applicare a `UndoHistory` i suoi passi attraverso le due porte (o una terza porta che faccia le stesse cose senza registrare l'annullamento), e rifiutare `undo` in un buffer in sola lettura.

BUF-2 — `cursor_pos()` costa O(posizione): ogni fotogramma rilegge il buffer fino al punto

Gravità	MEDIA
Dove	`buffer/gap_buffer.rs` (`cursor_pos`); la chiamano a ogni fotogramma `ui/compose.rs` (`LayoutNode::compute_tiled_views`), `ui/mode_line.rs` (`expand_mode_line`) ed `editor/frame.rs` (`EditorState::snapshot`, per una finestra flottante)
Verifica	Misurato nell'analisi di `08a853d` su un buffer di 8 MB: `snapshot` impiegava 7,9 ms con il punto alla fine e 0,03 ms con il punto all'inizio. `cursor_pos` non è cambiata da allora.

Cosa succede. Per trovare riga e colonna del punto `cursor_pos` scorre tutti i caratteri dall'inizio del buffer, mentre `cursor_1d_to_2d`, accanto, cerca già la riga nell'indice dei ritorni a capo. Ogni fotogramma la chiama almeno due volte per finestra (posizione del cursore e riga di stato), e circa trenta altri punti la usano.

Effetto. In un file grande ogni tasto costa millisecondi in più, e il costo cresce con la distanza del punto dall'inizio.

Correzione proposta. Ricerca binaria della riga nell'indice dei ritorni a capo, poi un conteggio dei soli caratteri della riga.

BUF-3 — `GapBuffer::clone` è `todo!()` e `find_forward` va in underflow su un buffer vuoto

Gravità	BASSA
Dove	`buffer/gap_buffer.rs` (`impl Clone for GapBuffer`, `find_forward`), `buffer/buffer_trait.rs` (`BufferTrait`, che richiede `Clone`)
Verifica	Dalla lettura del codice; oggi nessun percorso li raggiunge (`find_forward` è usata solo nei test).

Cosa succede. `BufferTrait` richiede `Clone`, ma l'implementazione per `GapBuffer` è un `todo!()`: il primo che clonerà un buffer manderà l'editor in panic. `find_forward` calcola `self.len() - 1`, che su un testo vuoto va in underflow (panic in debug) e poi arriva a un `unreachable!()`.

Correzione proposta. Derivare `Clone`, oppure togliere il vincolo dal trait; in `find_forward` usare `checked_sub`.

BUF-4 — In un buffer in sola lettura molti comandi non dicono nulla, e `kill-rectangle` conserva un blocco che non ha tolto

Gravità	BASSA
Dove	`primitives/killing.rs` (`cut_out`, e le kill che la usano), `primitives/verbs.rs` (`replace_span` e i verbi), `primitives/comments.rs` (`apply`), `primitives/rectangle.rs` (`cut`, `insert_at_edge`, `insert_block` e i comandi che li usano), `primitives/macros.rs` (`kmacro-insert-counter`)
Verifica	Riprodotto con un test temporaneo durante il passo 16 della documentazione, sul codice di `9fcd84b` con i passi 12–15, che non cambiano il comportamento. In un buffer in sola lettura `kill-line`, `kill-word`, `upcase-word`, `transpose-chars`, `join-line`, `comment-dwim` e `zap-to-char` non cambiano il testo e lasciano vuota l'area messaggi. `kill-rectangle` restituisce 3 e lascia il testo com'è, e dopo `(killed-rectangle)` restituisce le tre righe del blocco; `delete-rectangle`, `open-rectangle` e `string-rectangle` restituiscono 3 senza aver cambiato nulla; `kmacro-insert-counter` segnala l'errore «This buffer is read-only». Che il contatore sia già avanzato viene dalla lettura del codice.

Cosa succede. Le due porte rifiutano la modifica e restituiscono `false`, e dirlo all'utente tocca a chi le chiama, con `edited`, che scrive «Buffer is read-only». Lo fanno le primitive di `edits.rs`, `kill-region`, `yank`, `yank-pop` e `indent-line-to`. Le kill che passano da `cut_out` ne ricevono una stringa vuota e la danno all'anello, che la ignora; i verbi e i commenti rispondono nil; i comandi dei rettangoli scartano il risultato delle porte e rispondono con il numero di righe. `kill-rectangle` legge il blocco prima di cancellarlo, e lo conserva per `yank-rectangle` anche se la cancellazione è stata rifiutata. `kmacro-insert-counter` fa avanzare il contatore prima di inserire, e segnala un errore al posto del messaggio.
Effetto. (1) Un tasto che non fa nulla e non dice nulla: in una vista in sola lettura, come dired o l'aiuto, sembra un editor rotto e non un buffer protetto. (2) Dopo `kill-rectangle` in un buffer in sola lettura, `yank-rectangle` in un altro buffer inserisce un blocco che non è mai stato tolto, cioè lo duplica.
Correzione proposta. Far usare `edited` a tutti i comandi che modificano, a partire da `cut_out` e da `replace_span`; nei rettangoli fermarsi al primo rifiuto, senza conservare il blocco né spostare il punto; in `kmacro-insert-counter` inserire prima di far avanzare il contatore.

3 Tastiera, comandi e primitive dell'editor

CMD-1 — I caratteri non ASCII non si possono digitare

Gravità MEDIA
Dove `input.rs` (`fill_default_keymaps`, che lega a `self-insert` solo l'ASCII stampabile, e `Keymap::binding`, con il ripiego per i caratteri), `editor/keys.rs` (`EditorState::handle_key_event`)
Verifica Riprodotto sulla revisione `3d846f1`: premendo `è` l'area messaggi mostra «è is undefined»; `(define-key nil "è" 'forward-char)` ora restituisce `t`.

Cosa succede. La mappa globale lega a `self-insert` un tasto per ogni carattere da spazio a tilde; ogni altro carattere arriva alla risoluzione dei tasti senza un comando. Da `0a87fee` `parse_key` conta i caratteri e non i byte, quindi un carattere non ASCII si può legare, e una mappa può dare un comando a tutti i caratteri senza un legame, con `(define-key MAPPA "<char>" COMANDO)`. Ma lo fanno solo le mappe di vim: la mappa globale non ne ha uno.
Effetto. Con la configurazione predefinita l'editor è inutilizzabile per scrivere in italiano, francese, tedesco o in qualunque lingua con lettere accentate, e per i simboli tipografici (`€`, `°`, `«»`).
Correzione proposta. Dare alla mappa globale un comando per i caratteri, che inserisca il carattere premuto, come fa `self-insert-command` di Emacs.

CMD-2 — Mancano Canc, Inizio, Fine, PagSu, PagGiù e i tasti funzione

Gravità MEDIA
Dove `input.rs` (`KeyCode`, e `parse_key`, senza un nome per questi tasti), `src/tui/input.rs` (`translate_key`, che scarta tutto il resto); nella GUI, `src/gui/app.rs` (`EditorApp::native_key`) e `src/gui/keys.rs` (`special_key`)
Verifica Riprodotto sulla revisione `3d846f1`: `(define-key nil "<delete>" 'delete-char)`, e lo stesso con `"<home>"` e `"<f1>"`, danno `nil` e «Invalid key sequence» nel registro, dove la stessa riga per `<delete>` c'è già dall'avvio. Il resto dalla lettura del codice: l'enum ha solo `Char`, `Backspace`, `Enter`, `Esc`, `Tab`, le quattro frecce e un segnaposto `None`.

Cosa succede. I tasti non rappresentati vengono scartati dalla TUI con una riga nel registro («no translation for the key»), e `parse_key` non ha un nome per loro: il modulo `electric-pair` lega `<delete>` e fallisce a ogni avvio. La GUI (`src/gui/app.rs`) traduce da sé Inizio, Fine, Canc, PagSu e PagGiù in `C-a`, `C-e`, `C-d`, `M-v` e `C-v`, o nei tasti di vim nello stato normale: lì quei tasti funzionano, ma non si possono legare. I tasti funzione mancano in tutte e due. Le pagine `editing` e `keymaps` lo dicono tra i limiti.
Effetto. Tasti che ogni utente si aspetta (Canc in particolare) non fanno nulla nel terminale, e non si possono legare né lì né nella GUI.
Correzione proposta. Estendere `KeyCode` e `translate_key`, aggiungere i nomi (`<delete>`, `<home>`, `<f1>`…) a `parse_key`, e togliere dalla GUI le traduzioni fisse in favore di legami nelle mappe.

CMD-3 — Argomenti numerici enormi: allocazioni che terminano il processo e panic

Gravità MEDIA
Dove `primitives/indentation.rs` (`indent-line-to`), `primitives/args.rs` (`repeat_count`), `primitives/macros.rs` (`kmacro-insert-counter`)
Verifica Riprodotto sulla revisione `3d846f1`: `(kmacro-insert-counter 70000)` va in panic con «Formatting argument out of range»; `(forward-word 300000000)` invece si ferma subito alla fine del buffer. `indent-line-to` non è cambiata dall'analisi, che l'aveva trovata con il fuzzer delle primitive.

Cosa succede. Alcune primitive scritte in Rust usano un numero dell'utente come quantità da allocare, senza un limite e senza carburante. `indent-line-to` crea una stringa di spazi lunga quanto la colonna richiesta. `kmacro-insert-counter` passa la larghezza a `format!`, che oltre 65 535 va in panic. Da `cf0d8ac` i comandi sulle parole si fermano invece al bordo del buffer, e non ripetono più a vuoto il loro passo.
Come riprodurlo.

```
(indent-line-to 300000000000)     ; abort: memory allocation failed
(kmacro-insert-counter 65536)     ; panic: Formatting argument out of range
```

Effetto. Un panic nel thread dei comandi chiude l'editor e lascia il terminale in raw mode (TUI-2); un'allocazione rifiutata termina il processo senza nemmeno un panic.

Correzione proposta. Limitare larghezze e conteggi (per esempio a un milione) con un errore `args-out-of-range`, a partire da `repeat_count`; usare `try_reserve` per le allocazioni guidate dall'utente.

CMD-4 — Le combinazioni di modificatori in `define-key` sono limitate e un errore va solo nel registro

> **Gravità** BASSA
> **Dove** `input.rs` (`parse_key`), `primitives/keymaps.rs` (`define-key`)
> **Verifica** Riprodotto sulla revisione `3d846f1`: `"M-C-x"` e `"C-S-a"` danno `nil`, `"C-M-x"` dà `t`.

Cosa succede. `parse_key` riconosce un solo prefisso tra `C-M-`, `C-`, `M-` e `S-`, quindi `M-C-x` (stesso tasto di `C-M-x`) e le combinazioni con Shift non si possono scrivere. Una sequenza non valida o un modo inesistente fanno restituire `nil` a `define-key`, con il motivo scritto solo nel registro. La docstring di `define-key` e la pagina `keymaps` ora dicono quali combinazioni non sono tasti.

Correzione proposta. Analizzare i prefissi in un ciclo, in qualsiasi ordine, e segnalare l'errore con un errore Lisp.

CMD-5 — `self-insert` ignora l'argomento prefisso

> **Gravità** BASSA
> **Dove** `input.rs` (`fill_default_keymaps`: il legame è `(self-insert "x")`, una forma che fornisce già i suoi argomenti), `editor/keys.rs` (`EditorState::handle_key_event`), `primitives/edits.rs` (`self-insert`)
> **Verifica** Riprodotto sulla revisione `3d846f1`: `C-u 3 x` inserisce una `x`.

Cosa succede. Una forma che fornisce già i suoi argomenti non riceve l'argomento prefisso, e `self-insert` non lo legge. La pagina `editing` ora lo dice tra i conteggi; in Emacs le `x` sono tre.

Correzione proposta. Legare i caratteri a un comando che legge il tasto premuto (come `self-insert-command` di Emacs), oppure passare il prefisso alle forme di `self-insert`.

CMD-6 — Un conteggio negativo non inverte i comandi che cancellano

> **Gravità** BASSA
> **Dove** `primitives/args.rs` (`repeat_count`), e i comandi che lo usano: `delete-char` in `primitives/edits.rs`, `kill-word` e `backward-kill-word` in `primitives/killing.rs`, `kill-sexp` e `backward-kill-sexp` in `primitives/sexp.rs`
> **Verifica** Riprodotto sulla revisione `3d846f1`: in `foo bar`, `(kill-word -1)` dalla fine e `(delete-char -1)` da metà non cambiano il testo.

Cosa succede. `repeat_count` trasforma un conteggio negativo in 0, quindi i comandi che lo usano non fanno nulla, mentre in Emacs agiscono all'indietro. Due parti della scheda sono risolte: da `4506432` una risposta non numerica a una domanda numerica viene chiesta di nuovo, e da `cf0d8ac` gli spostamenti (caratteri, parole, righe, paragrafi, espressioni) usano `movement_count` e con un conteggio negativo vanno all'indietro.

Correzione proposta. Usare `movement_count` anche nei comandi che cancellano, cancellando all'indietro con un conteggio negativo.

CMD-7 — Su Windows un incolla tiene i ritorni a capo CR, e lì quattro test dell'incolla fallirebbero

> **Gravità** BASSA
> **Dove** `primitives/edits.rs` (la variante per Windows di `normalize_line_endings`, che usa `insert-pasted-text`), `tests/paste_tests.rs`
> **Verifica** Dalla lettura del codice: i test non girano su Windows, e la verifica dei passi compila soltanto per quel sistema.

Cosa succede. `insert-pasted-text` converte in LF ogni CRLF e ogni CR isolato prima di inserire, perché l'indice delle righe conta solo `\n`. Su Windows invece

`normalize_line_endings` restituisce il testo com'è, e nessun commento dice perché. La GUI passa a `handle_paste` gli appunti del sistema, che su Windows hanno di solito i fine riga CRLF. `paste_tests` non esclude Windows, e lì `a_paste_with_crlf_endings_becomes_lines`, `a_paste_with_bare_carriage_returns_becomes_lines`, `a_mix_of_endings_all_become_lines` e `a_trailing_carriage_return_opens_a_line` fallirebbero.

Effetto. Su Windows un testo di più righe incollato lascia un CR alla fine di ogni riga, e un testo con CR isolati diventa una riga sola.

Correzione proposta. Decidere che cosa vuole Windows: convertire come sugli altri sistemi, togliendo la variante, oppure scriverne il motivo e limitare i quattro test agli altri sistemi.

CMD-8 — `close-floating-window` senza finestre flottanti sposta il focus sulla finestra 0, anche quando non esiste

Gravità	BASSA
Dove	`primitives/windows.rs` (`close-floating-window`), `managers/windows.rs` (`Windows::set_focused`, che non controlla l'id)
Verifica	Riprodotto con un piccolo programma temporaneo durante il passo 17 della documentazione, sul codice di `9fcd84b` con i passi 12–16, che non cambiano il comportamento. Dopo `(split-window-right)` e `(delete-window)` resta la finestra 1, che ha il focus; `(close-floating-window)` restituisce nil, e da lì `(selected-window)` risponde 0, `(window-buffer)` nil e `(split-window-below)` nil, con `(count-windows)` sempre 1; `(other-window)` non sposta il focus. `(select-window 1)` lo riporta sulla finestra che c'è, e la divisione torna a funzionare.

Cosa succede. La primitiva toglie la flottante più recente e rende il focus alla finestra che questa ricorda. Quando non ce n'è nessuna, l'id da rendere manca e la primitiva mette il focus sull'id 0, che `Windows::set_focused` accetta senza guardare se esiste. La docstring dice che senza flottanti non fa nulla. In `debug.lisp`, `backtrace-dismiss` la chiama con il tasto `q` del modo del buffer `*Backtrace*`.

Effetto. Il focus resta su una finestra che non c'è: non si può dividere, e `other-window` non ne esce.

Correzione proposta. Non toccare il focus quando non c'è una flottante da chiudere, e controllare l'id in `set_focused`.

CMD-9 — In vim ogni sessione di Insert copia e confronta il buffer intero

Gravità	BASSA
Dove	`core/lisp/features/vim.lisp` (`vim--begin-insert`, che ricorda `(buffer-string)`, e `vim--finish-insert`, che lo confronta con `string-diff`)
Verifica	Misurato con un test temporaneo durante il passo 21 della documentazione, compilato in release: su un buffer di 100 KB `i`, `x` ed Esc costano 1,3 ms, su uno di 2 MB 47 ms, mentre un movimento costa 0,2 ms in tutti e due.

Cosa succede. Per sapere che cosa la sessione ha cambiato, e farlo ripetere a `.`, `vim--begin-insert` copia il testo del buffer intero in una proprietà del buffer, e `vim--finish-insert` ne fa un'altra copia e confronta le due con `string-diff`. Il costo è proporzionale al file, non a ciò che si è scritto, e lo pagano ogni `i`, `a`, `o`, `c` e Replace, anche quando la sessione non cambia nulla; la copia resta in memoria per tutta la sessione.

Effetto. In un file di qualche megabyte entrare e uscire da Insert ha un ritardo visibile, che cresce con il file: decine di millisecondi a 2 MB, circa duecento a 8 MB. Sui file piccoli non si nota.

Correzione proposta. Ricordare l'intervallo toccato, dal punto d'ingresso e dalle modifiche che le porte registrano (l'undo le conosce già), invece del testo intero; oppure confrontare solo le righe tra la prima e l'ultima modificata.

4 Minibuffer

MIN-1 — Due domande aperte insieme condividono `*Minibuffer*` e lasciano una finestra fantasma

Gravità ALTA
Dove `primitives/minibuffer.rs` (`default-minibuffer-prompt`, che usa sempre il nome `*Minibuffer*`), `editor/windows.rs` (`EditorState::open_floating_window`), `editor/buffers.rs` (`EditorState::close_buffer`, che chiude la **prima** finestra flottante che mostra il buffer)
Verifica Riprodotto sulla revisione `3d846f1` con la sequenza `M-x one`, `M-x two`, Escape: con i due prompt aperti ci sono due finestre flottanti su `*Minibuffer*` e `one` è sparito; dopo Escape il fuoco è su `*scratch*` e una finestra flottante su `*Minibuffer*` resta aperta.

Cosa succede. Ogni prompt crea il buffer `*Minibuffer*` e una finestra flottante che lo mostra. Un secondo prompt aperto mentre il primo è ancora aperto (un `M-x` dentro `M-x`, o un comando che fa una domanda mentre un prompt è aperto) riusa lo stesso buffer, quindi il testo del primo si perde e le due finestre mostrano lo stesso testo. Escape chiude il buffer e la **prima** finestra trovata, cioè quella del prompt esterno. Resta così a schermo la finestra del prompt interno, senza un buffer valido, e il fuoco torna a `*scratch*`. Il prompt successivo riusa il nome, e la finestra fantasma ne mostra il testo.
Effetto. Una finestra che non si può più chiudere copre parte dello schermo fino al riavvio, e la risposta che l'utente stava scrivendo nel prompt esterno è persa.
Correzione proposta. Nomi di buffer distinti per prompt (`*Minibuffer-1*`…) e una pila di prompt, oppure rifiutare un secondo prompt finché il primo è aperto, come fa Emacs senza `enable-recursive-minibuffers`. In `close_buffer`, chiudere **tutte** le finestre flottanti che mostrano il buffer.

MIN-2 — `C-g` dentro il minibuffer non annulla la domanda

Gravità MEDIA
Dove `core/lisp/base/common-keymaps.lisp` (`C-g` ⟶ `keyboard-quit`, anche nel minibuffer), `feature/minibuffer.rs` (`install_minibuffer`: la mappa di `minibuffer-mode` non lega `C-g`)
Verifica Riprodotto sulla revisione `3d846f1`: `M-x` poi `C-g` scrive «Quit» e il prompt resta aperto.

Cosa succede. `keyboard-quit` abbandona una sequenza di tasti a metà e azzera l'argomento prefisso, ma non sa nulla dei prompt aperti. La pagina `minibuffer` ora lo dice: Escape è la via d'uscita.
Effetto. Chi viene da Emacs preme `C-g` per uscire da una domanda e non ottiene nulla; il prompt resta aperto e cattura i tasti successivi.
Correzione proposta. Legare `C-g` a `minibuffer-cancel` in `minibuffer-mode`.

MIN-3 — Il contenuto del minibuffer viene sostituito senza passare dalle porte delle modifiche, e solo fino al punto

Gravità BASSA
Dove `feature/minibuffer.rs` (`set_minibuffer_content`), chiamata da `primitives/minibuffer.rs` (`minibuffer_complete`, `minibuffer_choose_completion`, `step_history`)
Verifica Riprodotto sulla revisione `3d846f1`: dopo un `M-x forward-char`, un nuovo `M-x`, `abc`, `C-a` e `M-p` lasciano nel minibuffer `forward-charabc`. Il resto dalla lettura del codice.

Cosa succede. La funzione che scrive la risposta proposta (completamento, cronologia) cancella il testo prima del punto e inserisce quello nuovo direttamente nel testo: quello che sta dopo il punto resta. Da `c850aee` chiama poi `changed`, quindi versione e cache sono a posto; il resto delle porte resta saltato: nulla per l'annullamento, nessun controllo della sola lettura, overlay e testo virtuale non spostati (BUF-1).
Effetto. Con il punto a metà della risposta, la cronologia non la sostituisce ma ci si mette davanti, e la risposta che parte è sbagliata.
Correzione proposta. Sostituire tutta la risposta, con `delete_range` e `insert_text`.

MIN-4 — Un `minibuffer-read` con MODE nil apre il prompt nel modo `nil`, dove Invio ed Esc non rispondono

Gravità BASSA
Dove `primitives/minibuffer.rs` (`default-minibuffer-prompt`, che prende ogni simbolo come il nome del modo)
Verifica Riprodotto con un test temporaneo durante il passo 18 della documentazione, sul codice di `71e1554`, che non cambia il comportamento. Dopo `(minibuffer-read "P:" (lambda (s) (setq got s)) nil nil nil "chiave")`, `(major-mode)` nel prompt risponde `nil`; `h`, `i` e Invio lasciano nel prompt `hi` e un a capo senza chiamare ON-CONFIRM, ed Esc non lo chiude. Con MODE omesso, o `'minibuffer-mode`, la stessa sequenza conferma.

Cosa succede. Il prompt predefinito legge MODE come il nome di un modo quando è un simbolo o una stringa, e nil è il simbolo `nil`: il buffer del prompt si apre nel modo `nil`, che non esiste, senza i tasti di `minibuffer-mode`. La docstring dice che MODE è `minibuffer-mode` se manca, e chi vuole dare KEY senza MODE passa nil, come si fa in Lisp con un argomento facoltativo. Nessun modulo fornito con l'editor lo fa.

Effetto. Il prompt non si conferma né si chiude con i suoi tasti, Invio vi scrive un a capo, e `C-g` non lo chiude (MIN-2): ne esce solo chi sa chiudere il buffer `*Minibuffer*` in un altro modo.

Correzione proposta. Prendere nil come assente, con `args::optional_name`, come fanno le altre primitive per un nome facoltativo, e un test con MODE nil e KEY.

MIN-5 — `minibuffer-choose-completion` scrive nel buffer corrente, anche quando non è il prompt

Gravità MEDIA
Dove `feature/minibuffer.rs` (`set_minibuffer_content`, che scrive nel buffer corrente), `primitives/minibuffer.rs` (`minibuffer-choose-completion`, che la chiama senza guardare se un prompt è aperto)
Verifica Riprodotto con un test temporaneo durante il passo 19 della documentazione, sul codice di `20f7221`, che non cambia il comportamento. In un buffer `f` con «hello world» e il punto dopo «hello », `(minibuffer-choose-completion "x")` lascia «xworld», e un `(undo)` dopo toglie l'inserimento di «hello world» da un testo che non lo contiene più: il buffer resta vuoto. Dopo `(set-buffer-read-only t)`, `(minibuffer-choose-completion "y")` lascia «y», dove `self-insert` è rifiutato.

Cosa succede. La primitiva mette il valore scelto «nel prompt» con `set_minibuffer_content`, che lavora sul buffer corrente e non su `*Minibuffer*`, e non guarda se un prompt è aperto. Fuori da un prompt cancella il testo prima del punto del buffer corrente e vi scrive il valore, saltando le porte delle modifiche come in MIN-3: niente annullamento, niente controllo della sola lettura. Nessun modulo fornito con l'editor la chiama fuori da un prompt: il presentatore di `completion.lisp` la riceve da `minibuffer-complete` e la chiama dai tasti della sua striscia.

Effetto. Una chiamata fuori da un prompt, da `M-:` o da un modulo, cancella parte di un buffer di file, anche in sola lettura, e la storia dell'annullamento non la riporta indietro: il prossimo `undo` lavora su posizioni che non corrispondono più.

Correzione proposta. Scrivere nel buffer `*Minibuffer*` per nome, e non fare nulla, o dare un errore, quando nessun prompt è aperto; con la correzione di MIN-3, la risposta intera e attraverso le porte.

5 File e salvataggio

FIL-1 — Un salvataggio fallito con `C-x C-s` non viene segnalato

Gravità	ALTA
Dove	`primitives/files.rs` (`save-buffer`, `save-buffer--overwrite`, e `write_buffer`, con cui salvano anche `save-some-buffers` e l'uscita, in `primitives/quit.rs`); per confronto `write-file`
Verifica	Riprodotto sulla revisione `3d846f1`: `save-buffer` in un buffer senza file lascia l'area messaggi invariata e scrive «No file associated with this buffer» solo nel registro. Il caso dell'analisi, un percorso in una directory inesistente, passa dallo stesso ramo, che non è cambiato.

Cosa succede. L'errore di `std::fs::write` viene passato a `log_diagnostic`, che scrive nel registro e non nell'area messaggi. `write-file`, al contrario, lo mostra. La docstring di `save-buffer` ora dice che il motivo di un fallimento va nel registro.
Effetto. Chi salva su un disco pieno, su un file senza permessi di scrittura o su un percorso che non esiste più crede di aver salvato. L'unico avviso arriva all'uscita, con la domanda «… is unsaved. Quit anyway?», e solo se il buffer è ancora segnato come modificato.
Correzione proposta. Mostrare nell'area messaggi sia «Wrote PERCORSO» sia «Cannot save PERCORSO: motivo», come fa `write-file`.

FIL-2 — I file vengono scritti sul posto: un errore a metà li lascia troncati

Gravità	MEDIA
Dove	`primitives/files.rs` (`save-buffer`, `write-file`, `save-buffer--overwrite`, `write_buffer`) e `jobs/autosave.rs` (`EditorState::write_auto_save`): tutti con `std::fs::write`
Verifica	Dalla lettura del codice.

Cosa succede. `std::fs::write` tronca il file e poi scrive. Se la scrittura si interrompe (disco pieno, processo terminato, alimentazione), sul disco resta un file troncato o vuoto, e la versione precedente è già persa.
Effetto. Possibile perdita del contenuto del file proprio quando qualcosa va storto.
Correzione proposta. Scrivere in un file temporaneo nella stessa directory, fare `sync_all` e poi `rename` sull'originale, conservando i permessi.

FIL-3 — Un file non UTF-8 non si apre e non viene detto nulla

Gravità	MEDIA
Dove	`editor/buffers.rs` (`EditorState::new_buffer`: `read_to_string`, errore solo nel registro), `primitives/files.rs` (`find-file`)
Verifica	Riprodotto sulla revisione `3d846f1` con un file Latin-1: area messaggi e buffer corrente invariati, `find-file` restituisce `nil`, e il registro dice «Error reading file: stream did not contain valid UTF-8».

Cosa succede. `std::fs::read_to_string` rifiuta qualsiasi byte non UTF-8; l'errore finisce nel registro e il comando non fa nulla di visibile. La docstring di `find-file` ora dice che un file che non si legge dà `nil`.
Effetto. File in Latin-1, file binari e file con un solo byte sbagliato sembrano non aprirsi «per nessun motivo».
Correzione proposta. Un messaggio nell'area messaggi; in prospettiva un'apertura con `String::from_utf8_lossy` in un buffer in sola lettura, o una decodifica Latin-1 di ripiego.

FIL-4 — Una modifica fatta durante il salvataggio può restare non salvata ma non segnata

Gravità	BASSA
Dove	`primitives/files.rs` (`save-buffer`, `save-buffer--overwrite`, `write_buffer`; `write-file` segue lo stesso schema)
Verifica	Dalla lettura del codice.

Cosa succede. Il testo viene copiato sotto il lock, poi scritto senza lock, poi `is_modified` viene azzerato con un secondo lock, senza guardare `version`. Una modifica arrivata nel frattempo (da un lavoro Lisp in background o da un callback) resta nel buffer ma il buffer risulta salvato. Il salvataggio automatico non ha il problema: ricorda la versione che ha copiato.

Correzione proposta. Ricordare la `version` copiata e azzerare `is_modified` solo se è ancora quella.

FIL-5 — Il salvataggio automatico segna come salvato anche un tentativo fallito

Gravità	BASSA
Dove	`jobs/autosave.rs` (`EditorState::write_auto_save`: `auto_saved_at` impostato anche dopo un errore)
Verifica	Dalla lettura del codice: il commento lo dice, «Recorded whether or not the write worked».

Cosa succede. Se la scrittura della copia fallisce per un motivo passeggero, la versione viene comunque registrata, e non si riprova finché il buffer non cambia. La prima metà della scheda è risolta da `1aa5cf7`: `auto_save_for` guarda file, sola lettura, modifica e versione già salvata prima di copiare il testo, e ogni giro guarda tutti i buffer e ne scrive al più quattro.
Correzione proposta. Registrare la versione solo dopo una scrittura riuscita.

FIL-6 — Con `auto-save-directory`, due percorsi diversi possono avere la stessa copia

Gravità	BASSA
Dove	`jobs/autosave.rs` (`flatten`, usata da `auto_save_path`)
Verifica	Riprodotto con un test temporaneo durante il passo 19 della documentazione, sul codice di `20f7221`, che non cambia il comportamento. `auto_save_path("/a!/b", Some("/d"))` e `auto_save_path("/a/!b", Some("/d"))` rispondono entrambe `/d/#!a!!!b#`.

Cosa succede. Per fare del percorso un solo nome di file, `flatten` cambia ogni separatore (`/`, `\`, `:`) in `!` e raddoppia ogni `!`. Così `/a!b/c` e `/a/b/c` restano distinti, ma un `!` accanto a un separatore no: un `!` seguito da un separatore e un separatore seguito da un `!` danno entrambi `!!!`. Il commento della funzione diceva che due percorsi diversi non possono coincidere. È lo schema di Emacs, che ha lo stesso limite. Senza `auto-save-directory` la copia sta accanto al file, con il solo nome, e il problema non c'è.
Effetto. Due buffer di file come `/a!/b` e `/a/!b` scrivono la stessa copia, e il ripristino di uno può dare il lavoro dell'altro.
Correzione proposta. Una codifica che due percorsi non possano condividere, come quella degli URL per il `%` e i separatori, o un hash del percorso nel nome; un test con i due percorsi di sopra.

6 Avvio, configurazione e registro

AVV-2 — Gli errori di sintassi nei file Lisp caricati vengono ignorati in silenzio

Gravità	MEDIA
Dove	`editor/loader.rs` (`EditorState::eval_file`: contenuto avvolto in `(progn …)`, errore del lettore → `Ok(nil)`; `EditorState::require`, che avvolge allo stesso modo); la usano `create_global_env`, per `init.lisp`, e le primitive `eval-file` e `require`
Verifica	Riprodotto sulla revisione `3d846f1` con `eval-file`: un file con una parentesi non chiusa e un file la cui ultima riga è un commento senza a capo restituiscono `nil` senza messaggi, e nessuna delle loro forme viene valutata.

Cosa succede. Il file viene letto come un'unica forma `(progn CONTENUTO)`. Una parentesi in più chiude la `progn` prima del tempo e il resto non viene mai letto. Una parentesi mancante, o un commento sull'ultima riga senza a capo (la parentesi aggiunta finisce **dentro** il commento), lascia la `progn` aperta. In entrambi i casi il lettore fallisce e `eval_file` restituisce `nil` senza dire nulla. `require`, dal passo 19b della documentazione, dà invece un errore quando il lettore fallisce; la parentesi in più, che il lettore non vede, tronca anche lì il file in silenzio, e il `provide` in fondo non arriva: l'errore dice che il file non ha fornito la feature.

Come riprodurlo.

```
init.lisp:
  (setq my-config-loaded 42)
  ;; fine della configurazione      ← senza a capo finale
→ dopo l'avvio my-config-loaded non è definita, e nessun messaggio
```

Effetto. L'utente vede l'editor partire senza la sua configurazione, o con metà della configurazione, e non c'è nulla che indichi perché.

Correzione proposta. Leggere le forme una alla volta e valutarle in sequenza, segnalando nell'area messaggi la riga del primo errore di sintassi. Come minimo, `format!("(progn\n{}\n)")`.

AVV-5 — La configurazione predefinita viene scritta solo se manca

Gravità	BASSA
Dove	`editor/boot.rs` (`DEFAULT_INIT_LISP`, e in `create_global_env` il blocco che scrive il file solo se manca), `paths.rs` (`app_dir`, per la directory della configurazione)
Verifica	Dalla lettura del codice.

Cosa succede. Il file `init.lisp` predefinito carica i moduli e imposta i legami. Una volta scritto non viene più toccato, quindi chi lo ha già non riceve i moduli aggiunti nelle versioni successive: dall'analisi se ne sono aggiunti quattro, `python-mode`, `toml-mode`, `bash-mode` e `vim`, che un `init.lisp` scritto prima non carica.

Correzione proposta. Separare la parte predefinita, caricata sempre dal codice o da un file di sistema, dalla parte personale, che è l'unica scritta nella directory dell'utente.

AVV-6 — Due editor aperti insieme scrivono lo stesso `rsedit.log` e si cancellano le righe a vicenda

Gravità	BASSA
Dove	`editor/log.rs` (`EditorState::enable_log_file`, con `File::create`), `managers/log.rs` (`Log::flush`, che scrive dalla posizione a cui era arrivato), `src/main.rs` (`main`, con un nome fisso nella directory della cache)
Verifica	Riprodotto sulla revisione `3d846f1` con due editor veri e la stessa directory di cache: all'avvio il secondo tronca il file e lo riscrive da capo; poi una riga del secondo («no translation for the key F(5)») e una del primo («Keymap not bound ») finiscono nello stesso punto, e il file dice «Keymap not bound » seguito dal resto dell'altra, «key F(5)».

Cosa succede. Il registro è sempre `rsedit.log` nella directory della cache, aperto con `File::create`, che tronca il file, e senza `O_APPEND`. Un secondo editor tronca quindi il file del primo e ci scrive dall'inizio; il

primo continua a scrivere dalla posizione a cui era arrivato, sopra le righe dell'altro. La pagina `diagnostics` dice che il file ricomincia a ogni avvio, non che due editor lo condividono.

Effetto. Con due editor aperti il registro su disco mescola le righe dei due e ne perde una parte: proprio il file che serve quando un editor si è chiuso da solo.

Correzione proposta. Un file per processo (con il PID nel nome, e un collegamento all'ultimo), oppure l'apertura in append con un blocco sul file, che dica al secondo editor che il registro è già in uso.

AVV-7 — Un errore in `init.lisp`, o in un modulo che richiede, impedisce all'editor di partire

Gravità	MEDIA
Dove	`editor/boot.rs` (`create_global_env`, che valuta `init.lisp` con `?`), `src/main.rs` (`main`, che stampa l'errore ed esce)
Verifica	Riprodotto durante il passo 19b della documentazione con l'editor vero, con `RSEDIT_CONFIG_DIR`, `RSEDIT_CACHE_DIR` e `RSEDIT_DATA_DIR` su tre directory temporanee: un `init.lisp` con `(require 'broken)`, e un `lisp/broken.lisp` che chiama una funzione che non c'è. L'editor stampa `UndefinedFunction("no-such-function")` ed esce con codice 0, senza aprire il terminale; il nome del modulo è solo nel log in memoria, che va perso.

Cosa succede. `create_global_env` valuta `init.lisp` con `?`, e il primo errore di valutazione, anche in un modulo che `init.lisp` chiede, risale fino a `main`, che lo stampa con `{:?}` e termina con `Ok(())`. Il log su file non è ancora aperto. Le pagine `configuration` e `diagnostics` dicevano che un modulo che fallisce lascia partire l'editor; lo fa solo un modulo che manca.

Effetto. Un errore in un modulo proprio o di un plugin, o in una riga di `init.lisp`, lascia senza editor finché non si corregge il file con un altro, e il messaggio non dice in quale file sta l'errore. Uno script che lancia l'editor lo crede uscito bene.

Correzione proposta. Valutare `init.lisp` forma per forma, o catturare l'errore in `create_global_env`, partire comunque e mostrare l'errore nell'area dei messaggi e nel log, come Emacs con `--debug-init` spento; in ogni caso uscire con un codice diverso da 0 quando l'editor non parte.

AVV-8 — Senza la directory della cache l'editor gira senza file di registro, e non lo dice

Gravità	BASSA
Dove	`src/main.rs` (`main`, che ignora l'errore di `app_dir` e quello di `enable_log_file`)
Verifica	Dalla lettura del codice, durante il passo 20 della documentazione.

Cosa succede. `main` attiva il file di registro solo se `app_dir` trova, o crea, la directory della cache, e scarta con `let _` l'errore di `enable_log_file`. In tutti e due i casi l'editor parte con il solo registro in memoria (`*Log*`), senza un messaggio nell'area dei messaggi né una riga nel registro stesso.

Effetto. Su una macchina con la cache non scrivibile, o con `RSEDIT_CACHE_DIR` sbagliata, il file che la pagina `diagnostics` indica non esiste, e chi lo cerca dopo un crash non trova nulla e non sa perché.

Correzione proposta. Scrivere nel registro in memoria, e nell'area dei messaggi, che il file non si è potuto aprire, con il percorso e l'errore.

7 Lavoro in background

BKG-2 — Un lavoro dell'editor che va in panic sparisce per tutta la sessione, e lo dice solo il registro

Gravità	BASSA
Dove	`work/core.rs` (`Core::run_step`: il lavoro che va in panic esce con `Exit::Panicked`), `editor/boot.rs` (`EditorState::new`, che sottomette evidenziatore e prescansione, e `create_global_env`, che sottomette controllo dei file e salvataggio automatico, una volta sola)
Verifica	Riprodotto sulla revisione `3d846f1` con un lavoro Lisp: dopo `(background-call 'boom (lambda () (set-face 'region "#aéb12")))` il registro dice «[BKG] panic: boom retired», il messaggio del panic va su stderr e un worker contatore continua a contare. Per i lavori dell'editor dalla lettura del codice.

Cosa succede. Da quando BKG-1 è risolta, un panic in un passo ritira solo il lavoro che l'ha causato e lo scrive nel registro con il tag `[BKG]`. Per un lavoro Lisp basta. I lavori dell'editor, evidenziatore, prescansione, controllo dei file e salvataggio automatico, sono però sottomessi una volta sola all'avvio, e un lavoro ritirato non torna: dopo un panic dell'evidenziatore la colorazione si ferma fino alla fine della sessione, e l'area messaggi non dice nulla. Un panic che avvenisse con un lock dell'editor in scrittura lo lascerebbe inoltre avvelenato, e il thread dei comandi andrebbe in panic al primo `.expect("write lock on …")`; oggi non si conosce un panic che avvenga sotto un lock.
Effetto. La colorazione, o il salvataggio automatico, smettono di funzionare senza che chi usa l'editor lo sappia.
Correzione proposta. Risottomettere i lavori dell'editor ritirati, con un limite perché un panic che si ripete non diventi un ciclo, e dire nell'area messaggi quale lavoro si è fermato; per i lock, decidere se uno avvelenato si può riprendere (`PoisonError::into_inner`) o deve chiudere l'editor in modo ordinato.

BKG-3 — `setenv` cambia l'ambiente del processo mentre il codice C di un altro thread può leggerlo

Gravità	BASSA
Dove	`primitives/environment.rs` (`setenv`)
Verifica	Dalla lettura del codice e della libreria standard di Rust.

Cosa succede. `std::env::set_var` e `remove_var` prendono un lock che prendono anche `std::env::var` e l'avvio di un processo con `std::process::Command`. Le letture dell'editor passano tutte di lì, anche sugli altri thread: un lavoro Lisp sull'esecutore che chiama `getenv` o `shell-command-to-string`, il salvataggio automatico che espande `~`, `stop shell` che avvia `/bin/sh` per il `KILL`. Il lock non copre il codice che legge l'ambiente da sé, con il `getenv` della libreria C, ed è per questo che `set_var` è `unsafe`. Nella TUI non c'è codice del genere su altri thread; nel front-end grafico le librerie di sistema che eframe carica (OpenGL, X11, Wayland) leggono variabili d'ambiente, e niente promette che non lo facciano su un loro thread. `setenv` stesso può girare sull'esecutore, chiamato da un lavoro Lisp. Fino al passo 17 della documentazione il commento SAFETY di `setenv` diceva che gli altri thread leggevano l'ambiente solo per avviare un processo dal thread dei comandi.
Effetto. Nel front-end grafico, un `setenv` mentre una libreria di sistema legge l'ambiente è comportamento indefinito: con glibc può far cadere l'editor. Non è stato osservato.
Correzione proposta. Tenere le variabili dei processi in una tabella dell'editor, letta da `getenv` e passata a ogni `Command` con `env_clear` ed `envs`, senza cambiare mai l'ambiente del processo: `setenv` non avrebbe più bisogno di `unsafe`.

8 Modi, sintassi e colorazione

SIN-1 — Una regione di sintassi che corrisponde alla stringa vuota blocca il thread di background

Gravità	MEDIA
Dove	`primitives/grammar.rs` (`add-syntax-region`, che non controlla i pattern), `modes/grammar.rs` (`highlight_line`, che non avanza dopo un'apertura, una chiusura o un escape vuoti)
Verifica	Riprodotto nell'analisi di `08a853d`: con BEGIN `"x*"` ed END `"y*"` un thread restava al 97 % di CPU e la colorazione di un file di una riga non finiva mai; con BEGIN vuoto e NESTABLE la memoria cresceva di circa 90 MB al secondo; con un pattern normale lo stesso file si colorava in 20 ms. Il ciclo di `highlight_line` non è cambiato da allora.

Cosa succede. `highlight_line` avanza la posizione al punto in cui finisce l'evento trovato. Per una regola protegge il caso vuoto (`pos = resume.max(pos + 1)`), per regioni ed escape no. Una regione che si apre e si chiude con due corrispondenze vuote nella stessa posizione si riapre all'infinito; una regione annidabile con un BEGIN vuoto continua a impilare stati.

Come riprodurlo.

```
(add-syntax-region 'rust-mode "x*" "y*" 'comment)
;; poi aprire un file .rs qualsiasi
;; errore di battitura realistico: BEGIN "#*" con END "$"
```

Effetto. Il passo dell'evidenziatore non finisce, e con lui si ferma l'esecutore del WorkManager: prescansione, controllo dei file, salvataggio automatico, worker Lisp e `background-call` restano fermi; i comandi di shell e le ricerche, che hanno un thread proprio, no. Il watchdog lo scrive nel registro («[BKG] stuck step: highlighter running for 1.0 s», poi a 2 s, a 4 s…). Oppure il processo esaurisce la memoria. Le grammatiche fornite con l'editor, comprese quelle nuove di Python, TOML, Bash e C, non hanno il problema; basta però un errore di battitura in una grammatica scritta dall'utente.

Correzione proposta. Rifiutare in `add-syntax-region` i pattern per cui `is_match("")` è vero, e in `highlight_line` forzare l'avanzamento di almeno un carattere dopo un evento vuoto.

SIN-2 — Colorare una riga molto lunga costa un tempo quadratico nella sua lunghezza

Gravità	MEDIA
Dove	`modes/grammar.rs` (`top_level_event`, che riprova ogni regione e ogni regola dalla posizione corrente, e `highlight_line`), `buffer/syntax.rs` (`LINES_PER_TURN`)
Verifica	Misurato nell'analisi di `08a853d` su una riga Rust ripetuta (`let a = "x";`): 65 KB in 0,96 s, 130 KB in 3,7 s, 260 KB in 14,8 s; una riga di 1 MB (un file minificato) richiederebbe circa quattro minuti. `top_level_event` non è cambiata da allora.

Cosa succede. A ogni evento la scansione chiede a ogni pattern la sua prossima corrispondenza a partire dalla posizione corrente. I pattern che non trovano nulla scorrono ogni volta fino alla fine della riga, quindi il lavoro è proporzionale a eventi × lunghezza. Una riga non si può dividere tra due turni, perché `LINES_PER_TURN` conta righe.

Effetto. Mentre l'esecutore del WorkManager colora la riga, prescansione, controllo dei file, salvataggio automatico e worker sono fermi, e il watchdog scrive nel registro un passo lungo e poi bloccato.

Correzione proposta. Ricordare per ogni pattern la prossima corrispondenza trovata e ricalcolarla solo quando la posizione la supera; oltre una certa lunghezza, non colorare la riga.

SIN-4 — `make-mode` su un modo esistente lo sostituisce con uno vuoto

Gravità	BASSA
Dove	`primitives/modes.rs` (`make-mode`), `editor/modes.rs` (`EditorState::set_mode`), `managers/modes.rs` (`Modes::insert`)
Verifica	Dalla lettura del codice.

Cosa succede. Rivalutare il file di un modo (per esempio dopo una modifica) azzera la sua mappa dei tasti, i suoi hook, la grammatica e la tabella di sintassi, compresi quelli che altri moduli avevano aggiunto (come l'hook di `electric-pair`). La docstring di `make-mode` e i limiti della pagina `writing-modes` ora lo dicono.
Correzione proposta. Se il modo esiste già, restituirlo senza modificarlo, oppure svuotare solo grammatica e tabella.

SIN-5 — Il registro delle facce cresce per sempre e l'indice è a 16 bit

Gravità	BASSA
Dove	`ui/faces.rs` (`Face::intern`, `Face::all`)
Verifica	Dalla lettura del codice.

Cosa succede. Ogni nome di faccia mai usato resta registrato. `(names.len() - 1) as u16` si avvolge dopo 65 536 nomi distinti, e un nome nuovo prenderebbe l'indice di uno vecchio. I nomi arrivano anche da Lisp mentre l'editor gira (`set-face`, `face-style`, le facce di overlay e testo virtuale), quindi il commento che dice il registro limitato dalla configurazione è ottimista.
Correzione proposta. Un controllo con errore oltre `u16::MAX`, o un indice a 32 bit.

SIN-6 — `forward-sexp` dall'interno di un simbolo salta alla fine dell'espressione successiva

Gravità	MEDIA
Dove	`modes/sexp.rs` (`forward`, `Scan::run_to`, `Scan::step`); la usano `forward-sexp`, `kill-sexp` e `backward-sexp` con un conteggio negativo
Verifica	Riprodotto sulla revisione `3d846f1`: in `(foo bar) (baz)`, da 2 (dentro `foo`) `forward-sexp` porta a 8 (fine di `bar`) invece che a 4; da 13 (dentro `baz`) non si muove invece di arrivare a 14.

Cosa succede. `forward` porta la scansione fino a `from` con `run_to`, poi conta le espressioni completate da lì in avanti. Ma un passo della scansione consuma un intero simbolo: se `from` cade dentro un simbolo, `run_to` lo completa (e lo scarta) prima di restituire il controllo, e il primo completamento contato è quello dell'espressione dopo. Sul simbolo finale di una lista non resta nulla da contare a quel livello e il punto non si muove.
Effetto. `C-M-f` e `kill-sexp` (che usa la stessa funzione) dall'interno di una parola si comportano in modo diverso da Emacs: il primo salta un'espressione, il secondo cancella anche l'espressione successiva.
Correzione proposta. In `run_to` fermarsi prima di un passo che supererebbe il limite, oppure in `forward` controllare se l'ultimo completamento di `run_to` finisce dopo `from` e contarlo.

SIN-7 — Una regola con GROUP colora tutta la corrispondenza quando il gruppo non vi prende parte

Gravità	BASSA
Dove	`modes/grammar.rs` (`top_level_event`, che ripiega sulla corrispondenza intera), `primitives/grammar.rs` (`add-syntax-rule`, che accetta un GROUP che il pattern non ha)
Verifica	Riprodotto con un test temporaneo durante il passo 19 della documentazione, sul codice di `20f7221`, che non cambia il comportamento. Dopo `(add-syntax-rule 'probe-mode "(foo)?bar" 'keyword 1)`, nella riga «foobar» è colorato solo «foo», ma la riga «bar» è colorata tutta come `keyword`. `(add-syntax-rule 'probe-mode "baz" 'string 5)` risponde t, e «baz» è colorata tutta come `string`.

Cosa succede. Per il testo da colorare la scansione prende il gruppo GROUP della corrispondenza, e se il gruppo non vi ha preso parte, o il pattern non lo ha, ripiega sulla corrispondenza intera. La docstring di `add-syntax-rule` dice «with GROUP, only that capture group of the match», e la pagina `writing-modes` «not the whole match»; il commento accanto al codice diceva che una regola così non colora nulla. Nessun modo fornito con l'editor ha una regola il cui gruppo sia facoltativo.
Effetto. Una regola scritta per colorare una parte facoltativa colora anche il resto della corrispondenza quando la parte manca.

Correzione proposta. Quando il gruppo non partecipa non colorare nulla, e scavalcare comunque la corrispondenza; in `add-syntax-rule` rifiutare un GROUP oltre i gruppi del pattern, come si rifiuta un REGEX sbagliato.

SIN-8 — `forward-sexp`, `kill-sexp` e `down-list` alla fine di una lista proseguono nelle liste che seguono

Gravità	MEDIA
Dove	`modes/sexp.rs` (`forward e down`, che non si fermano quando la lista di FROM si chiude); li usano `forward-sexp`, `kill-sexp`, `backward-sexp` con un conteggio negativo e `down-list`
Verifica	Riprodotto con un test temporaneo durante il passo 19 della documentazione, sul codice di `20f7221`, che non cambia il comportamento. Con la tabella di Lisp di `sexp_tests`, `forward` da 4 in `(a b) (c d)` risponde 8, la fine di `c`, e da 3 in `((a) (b c))` risponde 7, la fine di `b`; da 4 in `(a b)` e in `(a b) x` risponde 4. In un buffer in `risp-mode` con `(foo bar) (baz qux)` e il punto a 8, `(forward-sexp)` porta il punto a 14, dopo `baz`, e `(kill-sexp)` lascia `(foo bar qux)`. `down` da 2 in `(a) (b (c))` risponde 8, dentro `(c`, e in `(a) (b)` nil.

Cosa succede. `forward` conta le espressioni completate dopo FROM alla profondità di FROM, ma non si ferma quando la lista che contiene FROM si chiude: oltre la sua fine la scansione va avanti, e la prima espressione completata di nuovo a quella profondità, dentro una lista che segue, è contata. `down` cerca la prima volta che la profondità supera quella di partenza, e oltre la fine della lista la trova in una lista annidata in una che segue. La docstring di `forward-sexp` dice che alla fine di una lista il comando non fa nulla, e la pagina `editing` che `C-M-f` si ferma alla fine della lista; il test `at_the_end_of_a_list_forward_does_not_escape_it` lo prova solo con nulla dopo la lista.

Effetto. `C-M-f` alla fine di una lista salta dentro la lista seguente, e `kill-sexp` nello stesso punto cancella la chiusura della lista e l'inizio di quella dopo, e le fonde.

Correzione proposta. In `forward` e in `down`, fermarsi, con FROM o `None`, quando la profondità scende sotto quella di partenza; due test, con una lista dopo la fine di quella di FROM.

SIN-9 — `forward-sexp` si ferma alla fine di un commento a blocchi ma scavalca un commento di riga

Gravità	BASSA
Dove	`modes/sexp.rs` (`Scan::step`, ramo `Comment`: solo un commento a blocchi chiama `complete_comment`)
Verifica	Lo fissa il test `a_line_comment_is_skipped_but_a_block_comment_is_not` di `sexp_tests`, sul codice di `20f7221`: `forward` da 0 in `/* a */ b` risponde 7, la fine del commento, e in `// a\nb` risponde 6, dopo `b`.

Cosa succede. Un commento a blocchi completato è un'espressione, che `forward-sexp` raggiunge e su cui si ferma; un commento di riga si passa come uno spazio. Il test lo fissa dicendo che i due casi vanno resi uguali con una scelta esplicita. Emacs, con `parse-sexp-ignore-comments`, li scavalca tutti e due.

Effetto. Da prima di un commento a blocchi `C-M-f` si ferma alla sua fine; da prima di un commento di riga va oltre l'espressione che segue. `kill-sexp` cancella il commento solo nel primo caso.

Correzione proposta. Una regola sola per i due tipi, quella di Emacs, e il test cambiato con la scelta.

9 Ricerca, sostituzione e liste di risultati

RIC-1 — Una sostituzione con un'espressione che può essere vuota mette tutte le sostituzioni nello stesso punto

Gravità	MEDIA
Dove	`text/replace.rs` (`Replace::accept`), `editor/replace.rs` (`EditorState::replace_rest`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(replace-regexp "x*" "-")` su `abc` dà `-----abc` invece di `-a-b-c-`, e dice «Replaced 4 occurrences» dopo averne fatte cinque.

Cosa succede. Dopo una sostituzione di una corrispondenza vuota, la ricerca riprende da `found.start + new_len`, cioè subito dopo il testo appena inserito e **prima** del carattere successivo. Lì trova un'altra corrispondenza vuota, e così via. Si ferma solo per il tetto di sicurezza; il ciclo lo controlla dopo aver già sostituito, quindi le sostituzioni sono `len + 2` e il messaggio ne conta una di meno.
Correzione proposta. Per una corrispondenza vuota riprendere da `found.start + new_len + 1`, scavalcando un carattere, come fa Emacs; e contare le sostituzioni fatte.

RIC-2 — La ricerca all'indietro con un'espressione regolare costa un tempo quadratico

Gravità	MEDIA
Dove	`text/search.rs` (`Pattern::search_backward`, ramo `Regex`), usata da `feature/isearch.rs` (`scan`)
Verifica	Misurato nell'analisi di `08a853d` con `C-M-r` dalla fine di un file `ab ab ab …`, per il solo tasto `b`: 15 KB in 0,24 s, 30 KB in 0,99 s, 60 KB in 3,59 s. `Pattern::search_backward` non è cambiata da allora.

Cosa succede. Il motore delle espressioni cerca solo in avanti, quindi `search_backward` chiama `search_forward` in un ciclo e tiene l'ultima corrispondenza trovata. Ma ogni chiamata a `search_forward` copia l'intero buffer in una `String` e converte gli offset con una scansione lineare. È lo stesso costo quadratico che il commento sopra `Pattern::scan` dice di evitare.
Effetto. Su un file di qualche centinaio di KB con molte corrispondenze, ogni tasto premuto durante `C-M-r` blocca l'editor per minuti.
Correzione proposta. Una sola copia del testo e un solo passaggio con `captures_iter` che tiene l'ultima corrispondenza che finisce prima del limite.

RIC-3 — Ogni sostituzione è un gruppo di annullamento a sé

Gravità	BASSA
Dove	`editor/replace.rs` (`EditorState::replace_this`, che chiama `undo.boundary()` a ogni sostituzione)
Verifica	Riprodotto sulla revisione `3d846f1`: dopo `(replace-string "a" "b")` su `aaa`, che dice «Replaced 3 occurrences», un `undo` torna a `bba` e non ad `aaa`.

Cosa succede. Il confine di annullamento serve a `query-replace` per tornare indietro di una sostituzione, ma viene messo anche nelle sostituzioni non interattive.
Correzione proposta. Nessun confine tra le sostituzioni di `replace-string` e `replace-regexp`; un confine solo nel percorso interattivo.

RIC-4 — Limiti della ricerca confinata a una regione e delle scansioni su righe lunghe

Gravità	BASSA
Dove	`text/search.rs` (`Pattern::search_forward` con LIMIT, `Pattern::search_backward`, `Cursor::column`, `Pattern::scan`)
Verifica	Riprodotto sulla revisione `3d846f1`: su `aaaa` con la regione `[0, 2)`, `(replace-regexp "a+" "X")` risponde «No match for a+». Il costo della colonna viene dalla lettura del codice.

Cosa succede. Se la prima corrispondenza dopo il punto finisce oltre il limite, viene scartata invece di cercarne una più corta: `a+` su `aaaa` con la regione `[0, 2)` non trova nulla, anche se la regione contiene `aa`. Lo stesso vale per la ricerca incrementale all'indietro con un'espressione, che si ferma alla prima corrispondenza che supera il punto. Nella scansione completa (occur, grep), `Cursor::column` ricalcola i caratteri dall'inizio della riga per ogni corrispondenza, quindi una riga lunghissima con molte corrispondenze costa un tempo quadratico; grep però gira ora su un thread suo, e solo occur blocca l'editor.

Correzione proposta. Cercare nella sola fetta della regione; tenere la colonna in modo incrementale come già si fa per la riga.

RIC-5 — I risultati di occur, grep e compile puntano a righe che si spostano

Gravità	BASSA
Dove	`text/results.rs` (`Entry`, `Results::step`), `primitives/results.rs` (`visit`, `visit_index`, `refresh`)
Verifica	Riprodotto sulla revisione `3d846f1`: in `x`, `foo`, `bar`, dopo `(occur--scan "foo")` e una riga aggiunta in cima, `next-error` porta il punto all'inizio della riga di `x`.

Cosa succede. Ogni risultato conserva riga, colonna e offset come numeri, e `visit` si sposta per riga e colonna. Dopo una modifica che aggiunge o toglie righe sopra un risultato, `next-error` porta alla riga sbagliata; Emacs usa marcatori che seguono il testo. Il solo modo di aggiornare una lista, `results-refresh`, la ricostruisce da capo, e lo usa solo `compile` con l'uscita del compilatore.

Correzione proposta. Marcatori nei buffer aperti, aggiornati dalle due porte delle modifiche.

RIC-7 — Una lista di `M-s o` e una di `M-s g` aprono due strisce, una sopra l'altra

Gravità	BASSA
Dove	`editor/results.rs` (`display_results`, che cerca la striscia da riusare), `primitives/occur.rs` (`occur--scan` e `grep--scan`, che si presentano come produttori diversi)
Verifica	Riprodotto nell'editor sulla revisione `3d846f1`: una lista di `M-s o` e poi una di `M-s g` lasciano tre finestre, la lista di grep in una striscia nuova sopra quella di occur. Nessun test prova una lista di un tipo seguita da una dell'altro.

Cosa succede. `display_results` riusa la striscia in basso solo se la lista che vi è mostrata viene dallo stesso produttore, e occur e grep sono due produttori, `"occur"` e `"grep"`, anche se le loro liste sono entrambe in `occur-mode`. Prima di `3d846f1` una seconda lista, di qualunque dei due comandi, prendeva la striscia della prima.

Effetto. Chi cerca prima nel buffer e poi nella directory si ritrova con lo schermo diviso in tre, e deve chiudere la striscia in più con `q`. La pagina `searching` lo dice tra i limiti.

Correzione proposta. Se le due liste devono condividere la striscia, scegliere la striscia da riusare per tipo di lista (il modo del buffer) invece che per produttore, oppure dare a occur e a grep lo stesso produttore; in entrambi i casi un test con una lista di `M-s o` seguita da una di `M-s g`, e la riga tolta dai limiti della pagina.

RIC-8 — Un `query-replace` lasciato con un altro tasto resta aperto, con la corrispondenza evidenziata

Gravità	BASSA
Dove	`primitives/replace.rs` (`default_view`, che installa la domanda come mappa `OnUnbound::Release`), `editor/replace.rs` (`EditorState::end_replace`, che in quel caso nessuno chiama)
Verifica	Riprodotto con un test temporaneo durante il passo 13 della documentazione, sul codice di `9fcd84b` con i passi 12 e 13, che non cambiano il comportamento. Su `cat cat`, dopo `(query-replace "cat" "dog")` e il tasto `z`, la domanda sparisce e il testo è `zcat cat`; ma la sostituzione è ancora aperta, l'overlay `replace-match` copre ancora `cat` e `(replace-match)` risponde `(0 3 "zca" "dog")`. Dopo il tasto `y`, che ora inserisce una `y`, `(replace-this)` trasforma `zycat cat` in `dogat cat`.

Cosa succede. La vista predefinita installa la sua domanda come keymap transitoria `OnUnbound::Release`: un tasto che non è una risposta toglie la mappa e prosegue verso le altre keymap. La mappa se ne va dentro la risoluzione del tasto, che non ha un modo di chiudere ciò che c'è dietro, e nessuno chiama `end_replace`: la sessione resta in `Runtime`, l'overlay resta sul buffer, e le posizioni della sessione non seguono le modifiche successive. È la regola che `TransientKeymap` dichiara, una mappa con una sessione dietro è una `Refuse`, a non essere seguita.

Effetto. La corrispondenza resta evidenziata dopo che la domanda è sparita, finché un'altra sostituzione nello stesso buffer non toglie l'overlay; la pagina `searching` dice che un altro tasto ferma la sostituzione. I verbi della sessione restano attivi: chiamati da Lisp, per esempio da una vista scritta dall'utente, lavorano su posizioni vecchie, e `replace-this` sostituisce il testo sbagliato.

Correzione proposta. Chiudere la sessione quando la sua mappa se ne va: dopo ogni comando, fuori dalla risoluzione dei tasti, chiamare `end_replace` se una sostituzione è aperta e la sua mappa non è più installata. Oppure fare della domanda una mappa `Refuse` che lega anche un'uscita, rinunciando a lasciarla continuando a scrivere. In tutti e due i casi un test che, dopo il tasto in più, controlli la sessione e l'overlay e non solo la domanda.

RIC-9 — `results-put` e `results-select` prendono un BUFFER nil come il buffer chiamato `nil`

Gravità	BASSA
Dove	`primitives/results.rs` (`results-put e results-select`, che prendono ogni simbolo come il nome del buffer)
Verifica	Riprodotto con un test temporaneo durante il passo 18 della documentazione, sul codice di `71e1554`, che non cambia il comportamento. In `*scratch*`, `(results-put "p" "o" '(("buffer" "*scratch*" 1)) nil)` risponde 1, ma `(results-buffer)` e `(results-count "*scratch*")` rispondono nil, e `next-error` dice «No search results to walk»; senza il quarto argomento la lista si attacca a `*scratch*`. `(results-select nil)` risponde nil, dove `(results-select)` sceglie `*scratch*`.

Cosa succede. Le due primitive leggono BUFFER come un nome quando è un simbolo, e nil è il simbolo `nil`. `results-put` attacca la lista a un buffer `nil` che non c'è, cioè a nessuno, ne fa la lista che `next-error` percorre e risponde con il numero delle voci come se fosse andata; `results-select` cerca una lista nel buffer `nil`. Le docstring dicono il buffer corrente se manca, e le domande sulle liste (`results-count`, `results-entry`, `results-state`) prendono nil come assente. Nessun modulo fornito con l'editor passa nil.

Effetto. Un modulo che passa a `results-put` il suo argomento facoltativo perde la lista in silenzio, e la risposta dice il contrario.

Correzione proposta. Prendere nil come assente, come fanno le domande sulle liste, e un test per ciascuna primitiva.

10 Comandi di shell

SHL-4 — Il nome di una pagina di manuale viene passato alla shell senza virgolette

Gravità	BASSA
Dove	`core/lisp/features/manpage.lisp` (`manpage--system-page`, che passa a `shell-command-to-string` il comando `man` con il nome dentro; `manpage--rsedit-page`, che costruisce il percorso della pagina concatenando il nome)
Verifica	Riprodotto sulla revisione `3d846f1`: `(manpage--system-page "printf;touch FILE")` crea FILE.

Cosa succede. `(format "man -w %s 2>/dev/null" name)` passa il nome così com'è a `/bin/sh -c`. Un nome con spazi o metacaratteri viene interpretato dalla shell: `printf(3)` dà un errore di sintassi, che si presenta come «No manual entry», e `a;b` esegue due comandi. Il nome arriva senza filtri da `M-x manpage` e da `C-h m`; da `manpage-at-point` (e da `K` in vim) è limitato ai caratteri di un simbolo, che comprendono `*`, espanso dalla shell. Anche il percorso della pagina interna viene costruito concatenando il nome, quindi `../` esce dalla directory dei dati.

Correzione proposta. Racchiudere il nome tra apici singoli, con gli eventuali apici interni preceduti da escape, oppure rifiutare i nomi che non sono `[A-Za-z0-9._+-]+`.

SHL-5 — Un comando di shell che ignora `TERM` sopravvive all'editor

Gravità	BASSA
Dove	`primitives/shell.rs` (`stop_process`: il `KILL` dopo `GRACE`, 2 s, da un thread fuori dal WorkManager), `src/main.rs` (`SHUTDOWN_WAIT`, 500 ms, e il commento di `ShutDownOnExit`)
Verifica	Riprodotto con un piccolo programma temporaneo durante il passo 17 della documentazione, sul codice di `9fcd84b` con i passi 12–16, che non cambiano il comportamento: un editor avvia un comando, lo fa girare mezzo secondo, chiama `shutdown` con l'attesa di `src/main.rs` ed esce. Con `sleep 37` il processo non c'è più subito dopo; con `trap '' TERM; exec sleep 38` è ancora vivo tre secondi dopo l'uscita. Con l'editor aperto, `stop-worker` sullo stesso comando lo toglie entro tre secondi.

Cosa succede. Fermare un comando manda `TERM` e `CONT` al suo gruppo e avvia `stop shell`, un thread fuori dal WorkManager che dopo 2 s manda `KILL` a ciò che resta. All'uscita `shutdown` ferma i comandi così e aspetta al più 500 ms; poi il processo dell'editor finisce, e `stop shell` con lui, prima del `KILL`. Un comando che ignora `TERM`, o che lo gestisce senza uscire, resta vivo, in un gruppo che nessun terminale guarda. Il commento di `ShutDownOnExit` dice che un comando non sopravvive all'editor.

Effetto. Un processo dimenticato, che continua a usare CPU, file o porte dopo l'uscita.

Correzione proposta. All'uscita mandare `KILL` a ciò che resta dei gruppi dei comandi, dopo l'attesa di `shutdown`; oppure fare del `KILL` ritardato un lavoro del WorkManager, che `shutdown` aspetta.

11 Disegno e terminale

TUI-1 — I caratteri di controllo del testo arrivano al terminale: un file può comandarlo

Gravità	ALTA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Layout::visible_text`: copiano i caratteri senza filtrarli), `src/tui/render.rs` (`draw_clipped_row`, che li stampa con `Print`)
Verifica	Riprodotto nell'analisi di `08a853d`: in un file che contiene `ESC ] 52 ; c ; aGk= ESC \` la riga della fotografia conteneva la sequenza intatta, che la TUI scrive sul terminale. `ui/layout.rs` non è cambiato da allora, e il disegno della TUI, oggi in `src/tui/render.rs`, solo di posto.

Cosa succede. Tra il buffer e il terminale nessuno sostituisce i caratteri di controllo (ESC, BEL, CR, BS, DEL, i controlli C1). Il terminale li esegue. Passa dalla stessa strada l'uscita dei comandi, che da `02ea0e7` arriva decodificata dai byte così com'è. La GUI (`src/gui/paint.rs`) salta invece i caratteri di controllo.
Effetto. Aprire un file, guardare l'output di un comando, un risultato di grep o una pagina di manuale che contengono sequenze di escape fa eseguire quelle sequenze al terminale. OSC 52 riscrive gli appunti di sistema: l'utente incollerà poi in una shell un comando scelto da chi ha preparato il file. OSC 0 cambia il titolo della finestra; le sequenze CSI spostano il cursore, cancellano lo schermo o cambiano i colori. Anche senza malizia, un `\r` o un `\b` in un file rovinano la visualizzazione.
Correzione proposta. Al momento del disegno, sostituire i controlli (U+0000–U+001F tranne il tab, U+007F, U+0080–U+009F) con una rappresentazione visibile, come `^[` in Emacs o il carattere `␛`, contandone la larghezza nella mappa delle colonne.

TUI-2 — Un panic o un errore di I/O lasciano il terminale in raw mode

Gravità	ALTA
Dove	`src/tui/mod.rs` (`tui_main`), `src/main.rs` (`main`: nessun `panic::set_hook`, nessun `catch_unwind`; la guardia `ShutDownOnExit` ferma solo il lavoro in background)
Verifica	Dalla lettura del codice: nessun `set_hook`, e `catch_unwind` solo nel WorkManager. Un panic che lo innesca oggi è quello di TUI-5, riprodotto sulla revisione `3d846f1`.

Cosa succede. `tui_main` attiva il raw mode, il bracketed paste e, a richiesta, la cattura del mouse, e li disattiva solo sul percorso di uscita normale. Un panic nel thread dei comandi risale fuori da `main` senza ripristinarli; ogni `?` su un errore di I/O nel ciclo (`terminal::size`, `render_frame`, la cattura del mouse, un errore del thread che legge il terminale) esce da `tui_main` saltando il ripristino. Anche un panic che il WorkManager intercetta, in un lavoro di background, scrive il suo messaggio su stderr, sopra lo schermo.
Effetto. Dopo un crash la shell non mostra ciò che si digita, Invio non funziona e i clic del mouse stampano sequenze di escape. L'utente deve digitare `reset` alla cieca. Il messaggio del panic stesso viene stampato «a scalini», in raw mode.
Correzione proposta. Una struttura di guardia il cui `Drop` ripristina il terminale, più un `panic::set_hook` che ripristina prima di stampare il messaggio.

TUI-3 — Le tabulazioni non vengono espanse

Gravità	MEDIA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Row::to_screen`, `Row::to_buffer`: una colonna del buffer è una colonna dello schermo), `src/tui/render.rs` (`render_to`, `draw_clipped_row`)
Verifica	Riprodotto nell'analisi di `08a853d`: la riga della fotografia conteneva il carattere `\t`. `ui/layout.rs` non è cambiato da allora.

Cosa succede. L'editor conta un tab come una colonna. Il terminale invece lo espande fino alla tabulazione successiva, quindi il resto della riga viene disegnato più a destra di dove l'editor crede. Il commento in cima a `text/rectangle.rs` dice che un tab è una colonna, come in tutto l'editor: è il modello interno, non ciò che accade sullo schermo. Esiste ora un'impostazione `tab-width`, nel modulo `indent`, ma serve solo al rientro: il disegno non la legge.
Effetto. In un Makefile, in un file Go o in qualsiasi sorgente indentato con tab, il cursore appare nel punto sbagliato, i colori cadono sulle parole sbagliate e il resto della riga precedente può restare visibile.

Correzione proposta. Espandere i tab in spazi nella composizione delle righe (`compose_row`), con una larghezza configurabile, e usare la stessa mappa per il cursore e per i clic del mouse.

TUI-4 — I caratteri larghi contano come una colonna

Gravità	MEDIA
Dove	`ui/layout.rs` (`Row::plain`, `compose_row`, `Row::to_screen`, `Row::to_buffer`), `src/tui/render.rs` (`render_to`, che riempie le righe contando i caratteri)
Verifica	Dalla lettura del codice: le larghezze sono tutte `chars().count()`, e nessuna dipendenza calcola la larghezza di visualizzazione (`unicode-width`).

Cosa succede. Un ideogramma o un'emoji occupa due celle del terminale, ma l'editor ne conta una.
Effetto. Come per i tab: cursore, evidenziazioni, ritaglio e riempimento delle righe sfasati nei testi in cinese, giapponese, coreano o con emoji.
Correzione proposta. Calcolare la larghezza con `unicode-width` nella stessa mappa di TUI-3.

TUI-5 — Un colore esadecimale con caratteri non ASCII manda l'editor in panic

Gravità	MEDIA
Dove	`ui/faces.rs` (`Color::parse`), chiamata da `primitives/theme.rs` (`colour_arg`, dietro `set-face` e `define-face`)
Verifica	Riprodotto sulla revisione `3d846f1`: `(set-face 'region "#aéb12")` va in panic con «end byte index 2 is not a char boundary».

Cosa succede. `Color::parse` controlla la lunghezza in byte e poi taglia la stringa con `&hex[0..2]`. Con un carattere di due byte il taglio cade a metà del carattere.
Effetto. Chiusura dell'editor, con il terminale in raw mode (TUI-2). Il caso si presenta anche con un file di tema sbagliato. In un lavoro di background il panic ritira solo quel lavoro.
Correzione proposta. Rifiutare i colori che non sono ASCII (`hex.is_ascii()`) prima di tagliare.

TUI-6 — Le finestre flottanti non mostrano colori di sintassi né overlay

Gravità	BASSA
Dove	`editor/frame.rs` (`EditorState::snapshot`, il ciclo sulle finestre flottanti), a confronto con `ui/compose.rs` (`LayoutNode::compute_tiled_views`)
Verifica	Dalla lettura del codice: per una finestra flottante si raccolgono solo la regione e il testo virtuale.

Cosa succede. Un buffer mostrato in una finestra flottante appare senza colori e senza le evidenziazioni degli overlay; lo stesso buffer in una finestra affiancata le ha.
Correzione proposta. Usare per le finestre flottanti la stessa raccolta delle evidenziazioni delle finestre affiancate.

TUI-7 — `%p` nella riga di stato dice `All` solo per un buffer di una riga

Gravità	BASSA
Dove	`ui/mode_line.rs` (`position_in_buffer`, ed `expand_mode_line`, che non riceve l'altezza della finestra)
Verifica	Dalla lettura del codice.

Cosa succede. Il codice verifica `lines <= 1`: `expand_mode_line` non riceve l'altezza della finestra, e non può sapere se il buffer intero sta in una schermata, che è quando Emacs dice `All`. Il commento di `position_in_buffer` diceva il contrario; dal passo 20 della documentazione descrive il comportamento di oggi. Il test `the_position_reads_as_emacs_words_it` lo fissa.
Correzione proposta. Passare l'altezza della finestra e confrontare con quella.

TUI-8 — La TUI non usa lo schermo alternativo

Gravità	BASSA
Dove	`src/tui/mod.rs` (`tui_main`), `src/tui/render.rs` (`render_to`)
Verifica	Dalla lettura del codice: nessun `EnterAlternateScreen`.

Cosa succede. L'editor disegna sullo schermo principale del terminale e all'uscita lo cancella con `Clear(All)`.

Effetto. Il contenuto che il terminale mostrava prima di avviare l'editor, per esempio l'output di un comando, va perso.
Correzione proposta. `EnterAlternateScreen` all'avvio e `LeaveAlternateScreen` all'uscita, nella stessa guardia di TUI-2.

12 La finestra grafica

GUI-1 — La finestra grafica si aggiorna ogni 250 ms anche quando nulla cambia

> **Gravità** BASSA
> **Dove** `src/gui/app.rs` (`EditorApp::update`, `BACKGROUND_POLL`)
> **Verifica** Dalla lettura del codice, durante il passo 20 della documentazione.

Cosa succede. La TUI registra `on_background_change`, e l'editor la sveglia quando un lavoro di sfondo cambia qualcosa; la GUI no. Per vedere ciò che i lavori fanno chiede a egui un aggiornamento entro `BACKGROUND_POLL`, 250 ms, anche quando `next_redraw_in` risponde `None`. A ogni aggiornamento chiama `tick`, compone la fotografia con `snapshot` e la ridisegna, anche con l'editor fermo.
Effetto. Una finestra aperta e ferma consuma CPU, e un portatile la batteria, quattro volte al secondo; ciò che un lavoro di sfondo cambia compare fino a 250 ms dopo.
Correzione proposta. Registrare con `on_background_change` una callback che chiami `request_repaint` sul contesto di egui, e chiedere un aggiornamento solo quando `next_redraw_in` o `drag_scroll_in` danno una durata.

13 Documentazione, commenti, codice morto, build e test

DOC-1 — Commenti e docstring che descrivono un comportamento che non c'è più

Gravità	BASSA
Dove	vedi l'elenco
Verifica	Dalla lettura del codice.

Cosa succede.

- `primitives/shell.rs` (`shell-command-start`): il messaggio «The editor's background work has shut down» vale anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta. Il commento accanto, che diceva lo stesso, è corretto dal passo 17 della documentazione.
- `primitives/occur.rs` (`grep--scan`): il messaggio «The editor's background work has shut down» vale anche quando il sistema non dà un thread nuovo, l'altro caso in cui `WorkManager::spawn` rifiuta. Il commento accanto, che diceva lo stesso, è corretto dal passo 18 della documentazione.
- `primitives/workers.rs` (`define-worker` e `background-call`, attraverso `start`): la riga del registro «the editor's work has shut down» vale anche quando l'esecutore non è partito, l'altro caso in cui `WorkManager::submit` rifiuta. Il commento accanto è corretto come quello di `shell.rs`.

La voce su `src/tui.rs` (`render_to`, il commento sull'area messaggi, che diceva il minibuffer agganciato alle ultime righe del frame) è corretta dal passo 20 della documentazione, e il commento è oggi in `src/tui/render.rs`. Le voci dell'analisi su `primitives/workers.rs` e su `shell-command-to-string` non valgono più: il codice e la docstring sono stati riscritti (`6de16a1`, `4f05134`). Le tre sui commenti della libreria di base dell'interprete sono nel rapporto di risp, dentro le schede INT-9, INT-16 e INT-21.

Correzione proposta. Aggiornarli insieme alle correzioni dei problemi a cui rimandano.

DOC-2 — Trappole latenti: `Debug` e `PartialEq` di `EditorState`

Gravità	BASSA
Dove	`editor/mod.rs` (`impl Debug for EditorState`, e accanto `impl PartialEq for EditorState`)
Verifica	Dalla lettura del codice.

Cosa succede. `Debug` per `EditorState` è un `todo!()` e `PartialEq` un `unreachable!()`: li richiede il trait `LispContext` di risp. Un `dbg!`, un `{:?}` o un `assert_eq!` fallito su una struttura che contiene uno stato dell'editor va in panic invece di stampare; oggi nessun tipo di risp contiene un contesto, e la trappola resta latente.

Correzione proposta. Un `Debug` che stampi almeno i nomi dei buffer, e un `PartialEq` per identità.

DOC-5 — Il codice delle liste di risultati lascia un metodo morto, due `if` da unire e righe da formattare

Gravità	BASSA
Dove	`editor/buffers.rs` (`free_buffer_name`), `editor/results.rs` (`select_results_target`, `quit_results`), e i file del codice toccato da `3d846f1`: `editor/results.rs`, `editor/settings.rs`, `editor/boot.rs`, `primitives/results.rs` e `primitives/occur.rs`, in cui il passo 18 della documentazione lo ha diviso, `primitives/shell.rs`
Verifica	Riprodotto sulla revisione `3d846f1`: ogni `cargo build` avvisa che `free_buffer_name` non è mai usato; `cargo clippy` dà tre avvisi in più della revisione precedente, `5f79477`; `cargo fmt --check` trova 41 blocchi da riformattare in quei cinque file, dove prima ne trovava 3, tutti in `src/gui.rs`.

Cosa succede. Con `3d846f1` le liste di occur si aprono con `prepare_results_buffer`, che sceglie il nome con `free_name_for_work`: se ne va così l'ultima chiamata a `free_buffer_name`, e il metodo resta, con la sua docstring. In `select_results_target` e in `quit_results` un `if let` contiene un solo `if`, che clippy chiede di unire (`collapsible_if`). Il codice nuovo non è passato da `rustfmt`: righe spezzate dove ci starebbero intere, la registrazione di `results-display` e `results-quit` con l'indentazione persa, spazi in fondo a una riga di `primitives/shell.rs`.

Effetto. Nessuno per chi usa l'editor. Un avviso fisso in ogni compilazione nasconde quelli nuovi, e la prossima modifica a questi file, passata da `rustfmt`, porterà con sé decine di righe che non c'entrano.
Correzione proposta. Togliere `free_buffer_name`, unire i due `if`, passare `cargo fmt` sui cinque file, in un commit che non cambia altro.

14 Appendici

14.1 Le schede risolte

Le schede risolte dopo l'analisi, con il commit che le ha risolte. Il testo completo di ciascuna sta nella storia di git di questo file; che cosa è cambiato, e perché, nel messaggio del commit.

ID	Problema	Gravità	Risolto in
AVV-1	L'editor non parte se la directory corrente non è scrivibile	ALTA	`58d0b0d`
AVV-3	Un errore di scrittura del registro manda l'editor in panic	MEDIA	`a20f050`
AVV-4	Il registro in memoria cresce senza limite	BASSA	`a20f050`
BKG-1	Un panic in un lavoro di background uccide per sempre il thread condiviso	ALTA	`6de16a1`
SIN-3	`write-file` che cambia il modo lascia i colori del modo precedente	BASSA	`cf0d8ac`
RIC-6	Alla fine di una ricerca incrementale il mark non è dove la ricerca è cominciata	BASSA	`1aa5cf7`
SHL-1	Un comando che scrive molto su stderr si blocca	MEDIA	`02ea0e7`
SHL-2	Un solo byte non UTF-8 nell'output interrompe il comando	MEDIA	`02ea0e7`
SHL-3	Non c'è modo di fermare un comando in corso	BASSA	`02ea0e7`
DOC-3	Lo script di build presume dove si trova la directory `target`	BASSA	`f742701`
DOC-4	Un test di prestazioni instabile e clippy che fallisce sui test	BASSA	`10598fa`

14.2 Differenze da Emacs che non sono difetti

Alcuni comportamenti differiscono da Emacs per scelta, o comunque senza produrre risultati sbagliati. Sono elencati qui perché chi conosce Emacs non li scambi per errori. Quelle dell'interprete sono nel rapporto di risp.

- Le espressioni regolari usano la sintassi del crate `regex` di Rust, non quella di Emacs. `\(` è una parentesi letterale e i gruppi si scrivono `( )`. Non esistono lookaround né riferimenti all'indietro, ed è proprio ciò che garantisce un tempo lineare.
- Le liste di risultati, le ricerche e le sostituzioni non tornano mai all'inizio da sole: quando la lista finisce lo dicono, come scelta esplicita di progetto.

14.3 Parti esaminate e trovate solide

Nell'analisi queste parti sono state esercitate di proposito sui casi limite senza trovare problemi:

- lo scanner delle espressioni bilanciate (`modes/sexp.rs`): stringhe con escape alla fine del buffer, parentesi sbilanciate in entrambe le direzioni, letterali carattere, lifetime di Rust, stringhe raw, commenti a blocco annidati, commenti che contengono parentesi. Fuori da questi casi, più tardi, le verifiche hanno trovato SIN-6 e SIN-8
- la ricerca incrementale: parentesi letterali, espressioni non valide, cancellazione, ricerca fallita e ripresa dopo il fallimento;
- kill e yank, compreso `M-y`, e le macro di tastiera (con il limite alla ricorsione delle macro);
- `kill-buffer` di un buffer mostrato in due finestre, la chiusura dell'unica finestra, la chiusura di `*scratch*`;
- la sostituzione `a` → `aa`, che termina;
- i comandi di modifica ai bordi del buffer;
- `delete-file` sui link simbolici a directory, che cancella il link e non il contenuto della directory di destinazione;
- l'uscita con buffer non salvati: dopo un salvataggio fallito il buffer resta segnato come modificato e l'editor chiede conferma prima di uscire.